



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



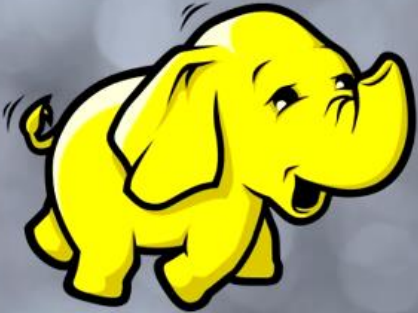
Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Hadoop Architecture – History and Evolution

Hadoop is a distributed data processing platform that offers three core capabilities listed below:

1. YARN
2. HDFS
3. Map/Reduce



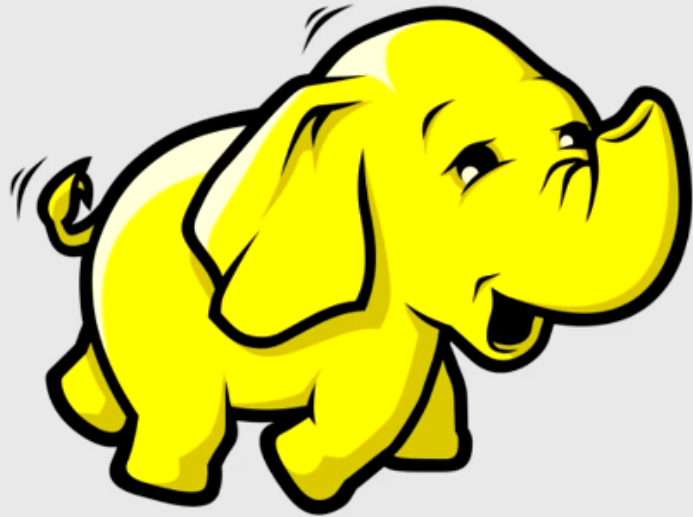
Hadoop Platform

Hadoop is a distributed data processing platform that offers the following core capabilities.

-  YARN - Cluster Resource Manager
-  HDFS - Distributed Storage
-  Map/Reduce – Distributed Computing

YARN:

YARN is the Hadoop cluster resource manager. It allows multiple applications to run on the Hadoop Cluster and share resources amongst the applications.



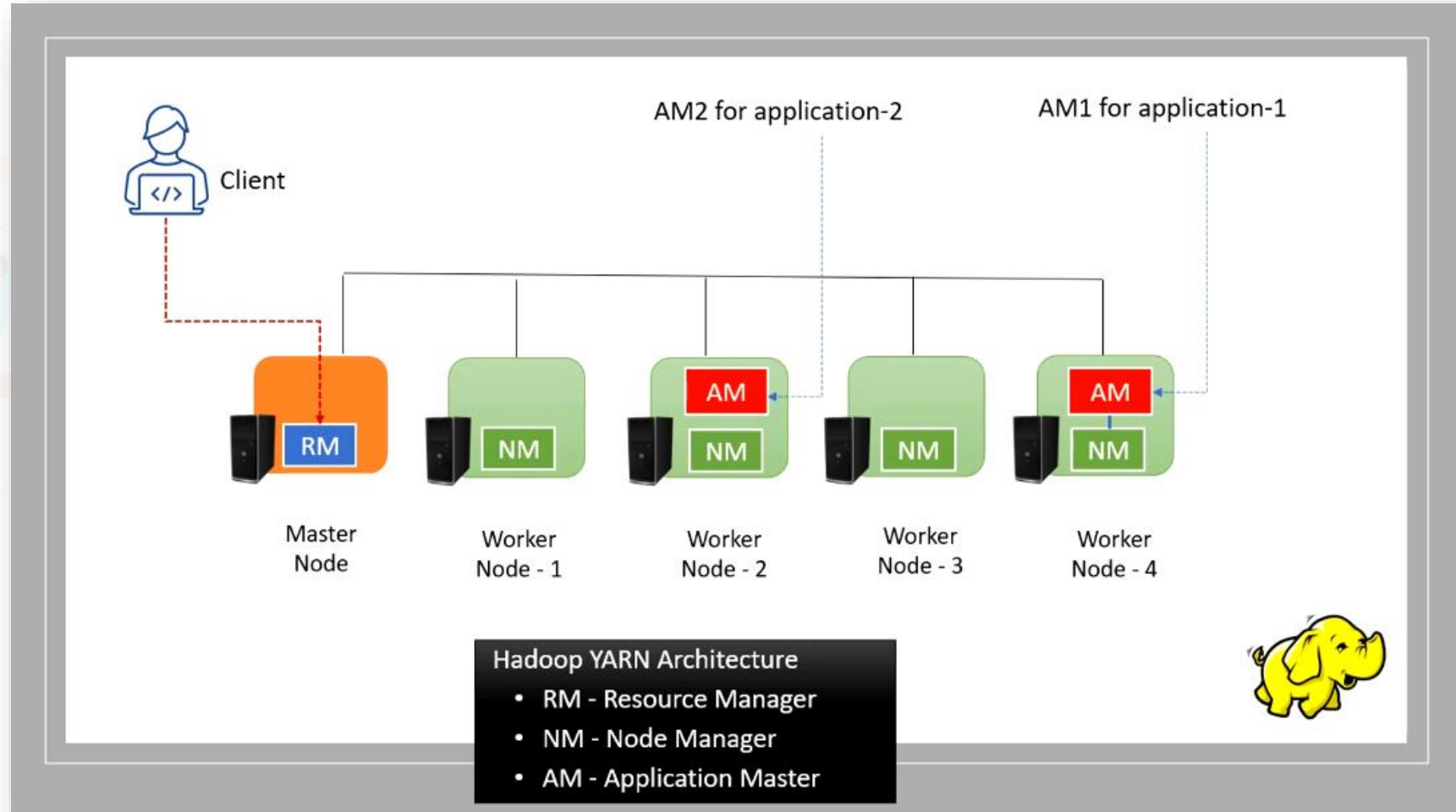
YARN – Yet Another Resource Manager

- Hadoop cluster Operating System
- Popularly known as Hadoop Cluster Resource Manager
- Has three main components
 - RM - Resource Manager
 - NM - Node Manager
 - AM - Application Master

YARN:

Assume we installed Hadoop, and now these five computers form a Hadoop cluster. Hadoop uses a master-slave architecture.

So one of these machines will become the master, and the remaining will act as the worker node.



YARN:

We have a five-node cluster that I showed you in the earlier slide. One node acts as a master and runs the YARN resource manager service. The other four nodes act as a Worker and run a node manager service.

The node manager will regularly send the node status report to the resource manager. We created a Hadoop cluster so we can run big data applications. For running an application on Hadoop, you must submit the application to the YARN resource manager. Assume you submitted a Java application to the YARN using a command line submit tool. Now the resource manager should run this application on the cluster.

So, the resource manager will request one of the node managers to start a resource container and run an AM (application master) in the container. And your application starts running inside the Application Master container.

So, we submit our application to the Resource Manager.

The resource manager requests the node manager for allocating an application master container and starting your application inside the AM container. Each application on YARN runs inside a different AM container.

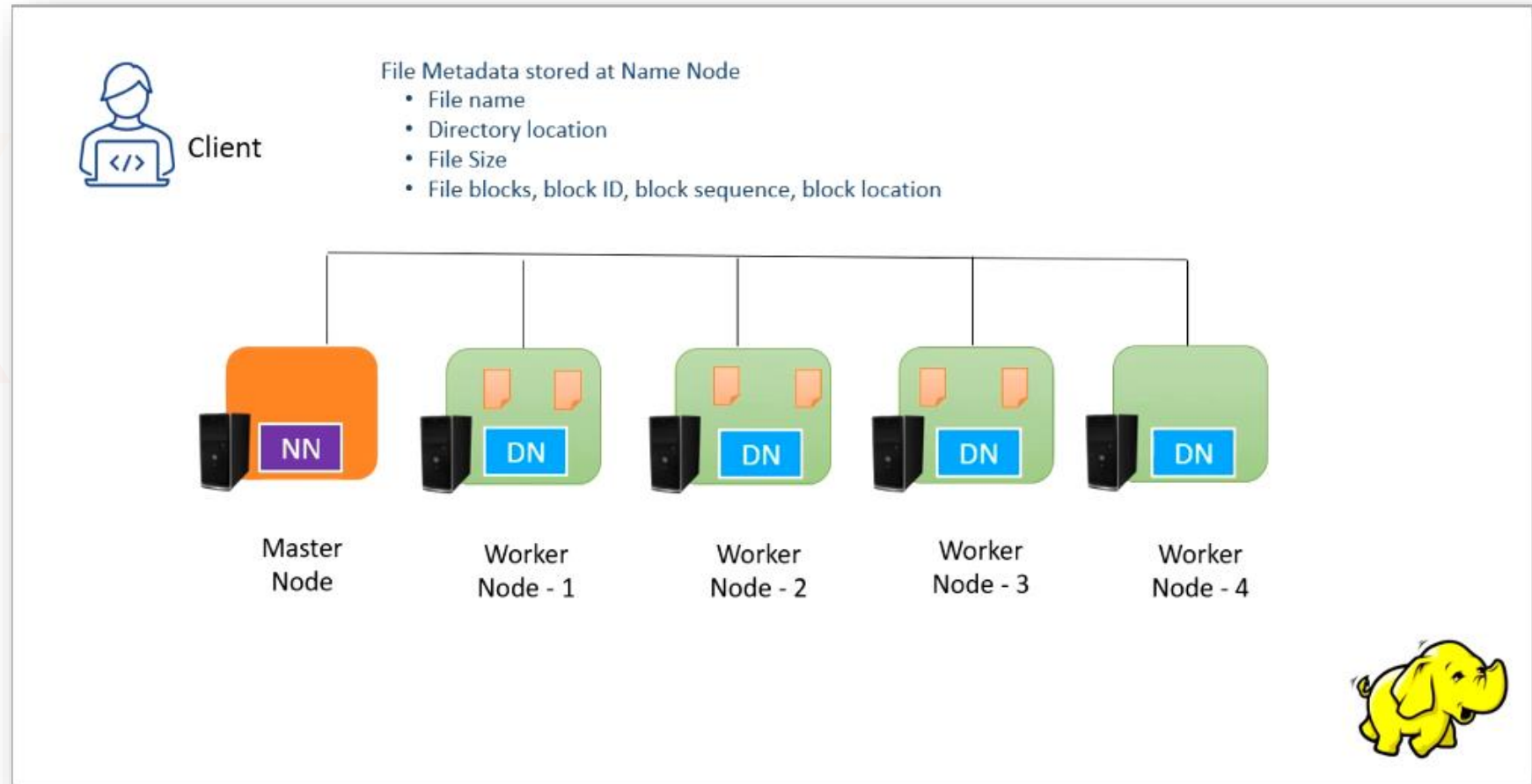
If you have ten applications running in parallel, you will see 10 AM containers on your Hadoop cluster.

HDFS:

The HDFS stands for Hadoop Distributed File system, and it allows you to save and retrieve data files in the Hadoop Cluster.

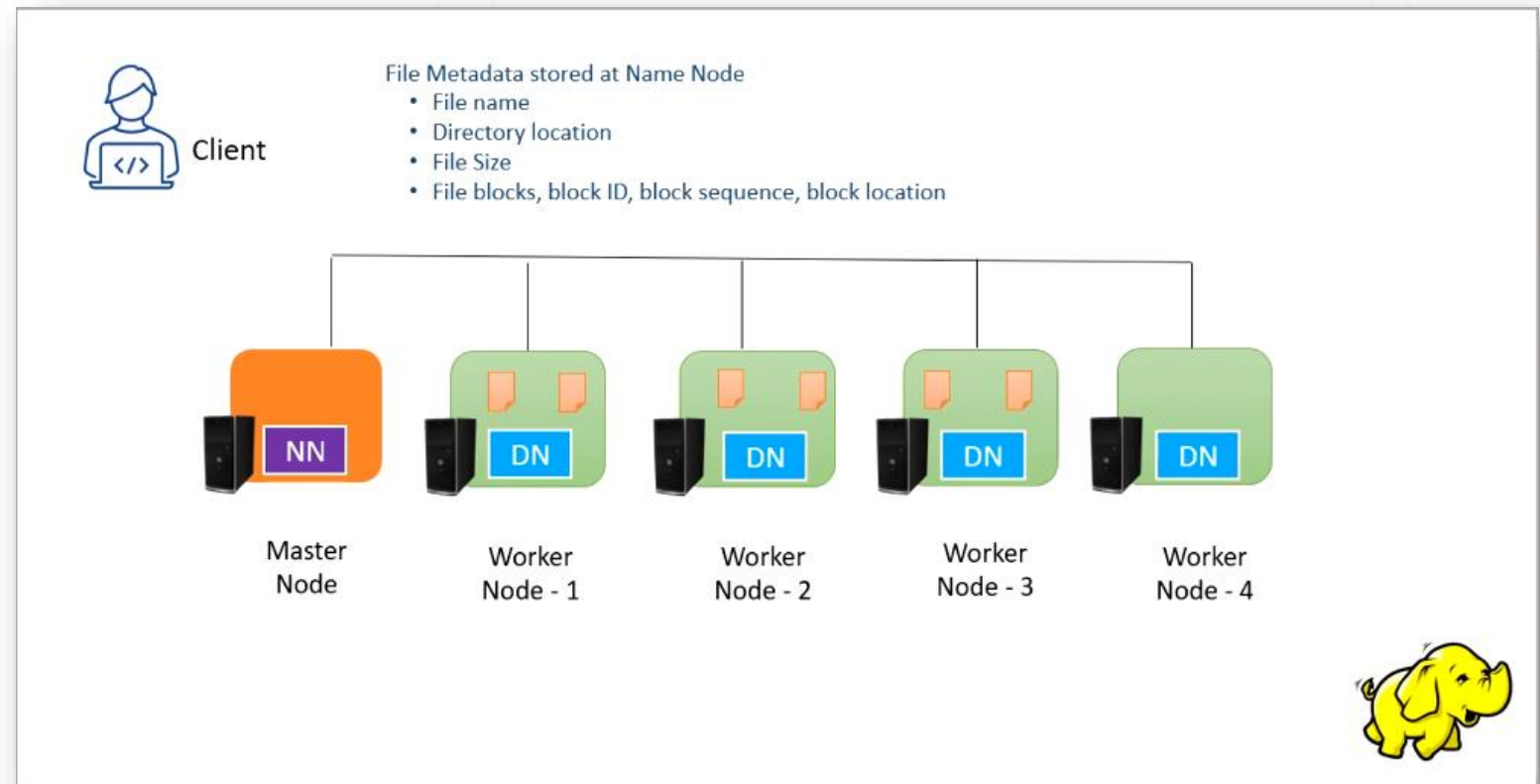
The HDFS has the following components.

1. Name Node (NN)
2. Data Node (DN)



HDFS:

Assume we have five computers shown below. We already installed Hadoop on these computers and created a Hadoop cluster. Hadoop will install the Name Node service on the master. And each worker node runs a data node service. The name node with the data node service forms the HDFS. The primary purpose of the HDFS is to allow us to save files on this Hadoop cluster and read them whenever required.



Map/Reduce:

Map-reduce is a programming model and a framework. A programming model is a technique or a way of solving problems. The M/R framework is a set of APIs and services that allow you to apply the map-reduce programming model.

Hadoop taught us the map-reduce programming modal and also offered a Map-Reduce programming framework to implement it.

Map/Reduce:

You have to count the lines in a 20 TB CSV file. There are two challenges in this problem statement:

1. Huge file size, It is hard to find machines to store 20 TB of data. And this problem becomes more complex if we grow the size in petabytes.
2. We also have a processing time problem. A simple line count on a 20 TB file takes hours or days.

Problem Statement

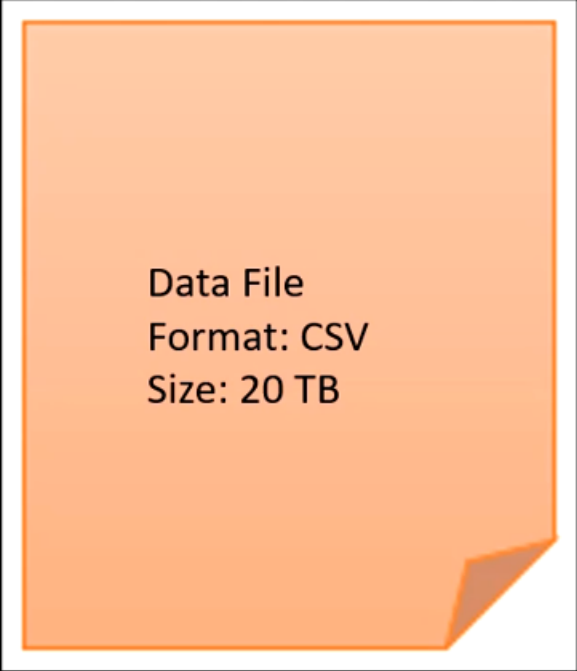
Count the lines in the given file

Solution Pseudocode

```
open file as f_hd
for each t_line in f_hd.get_line()
    n_count = n_count + 1
close f_hd
print n_count
```

Challenges

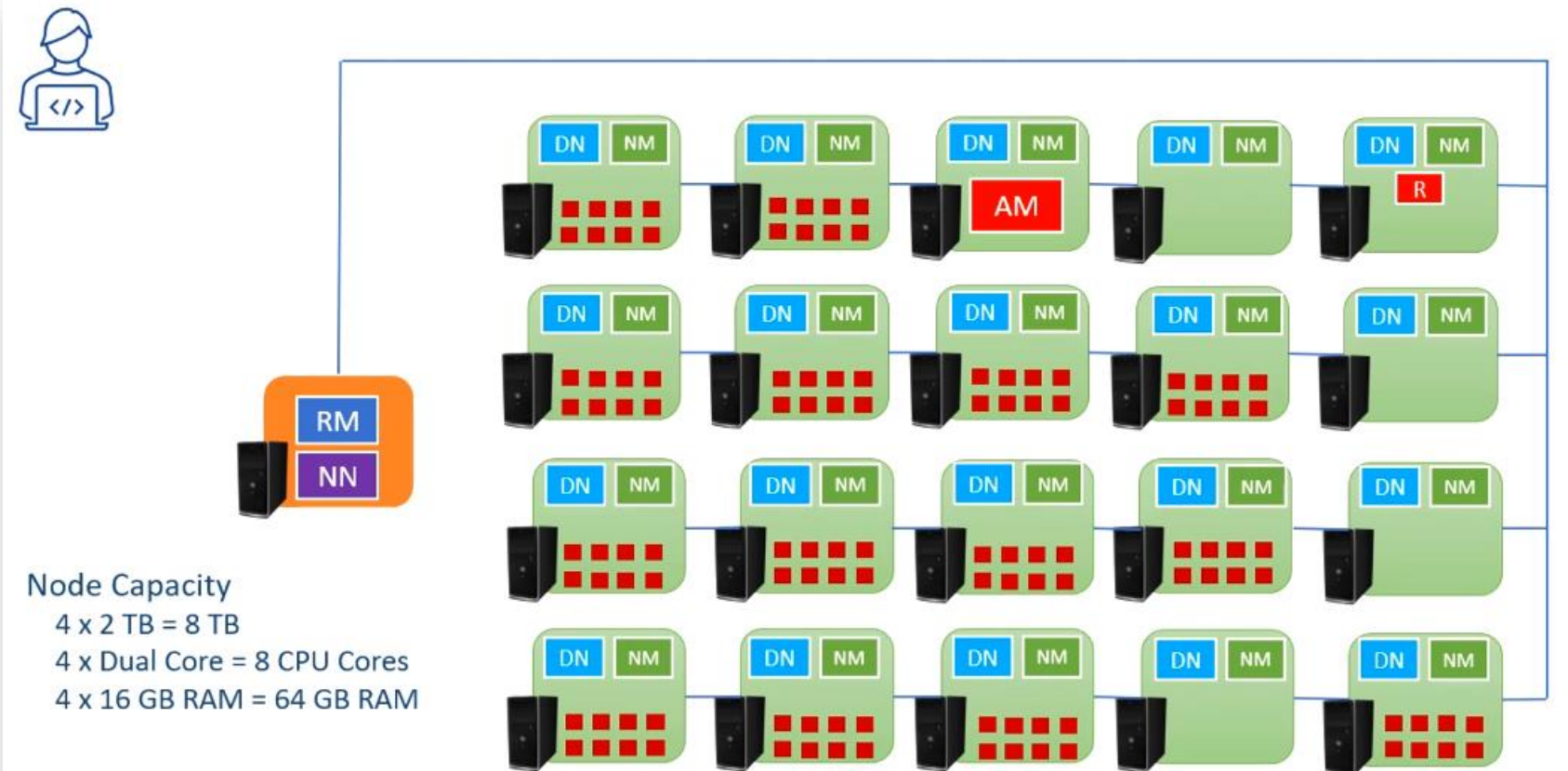
1. Storage capacity
2. Processing time

A diagram of a data file represented as an orange rectangular card with a folded bottom-right corner. The card is placed on a black background with a white border. To the left of the card are four white diagonal slashes. To the right is a white circle. The card contains the text: Data File, Format: CSV, Size: 20 TB.

Data File
Format: CSV
Size: 20 TB

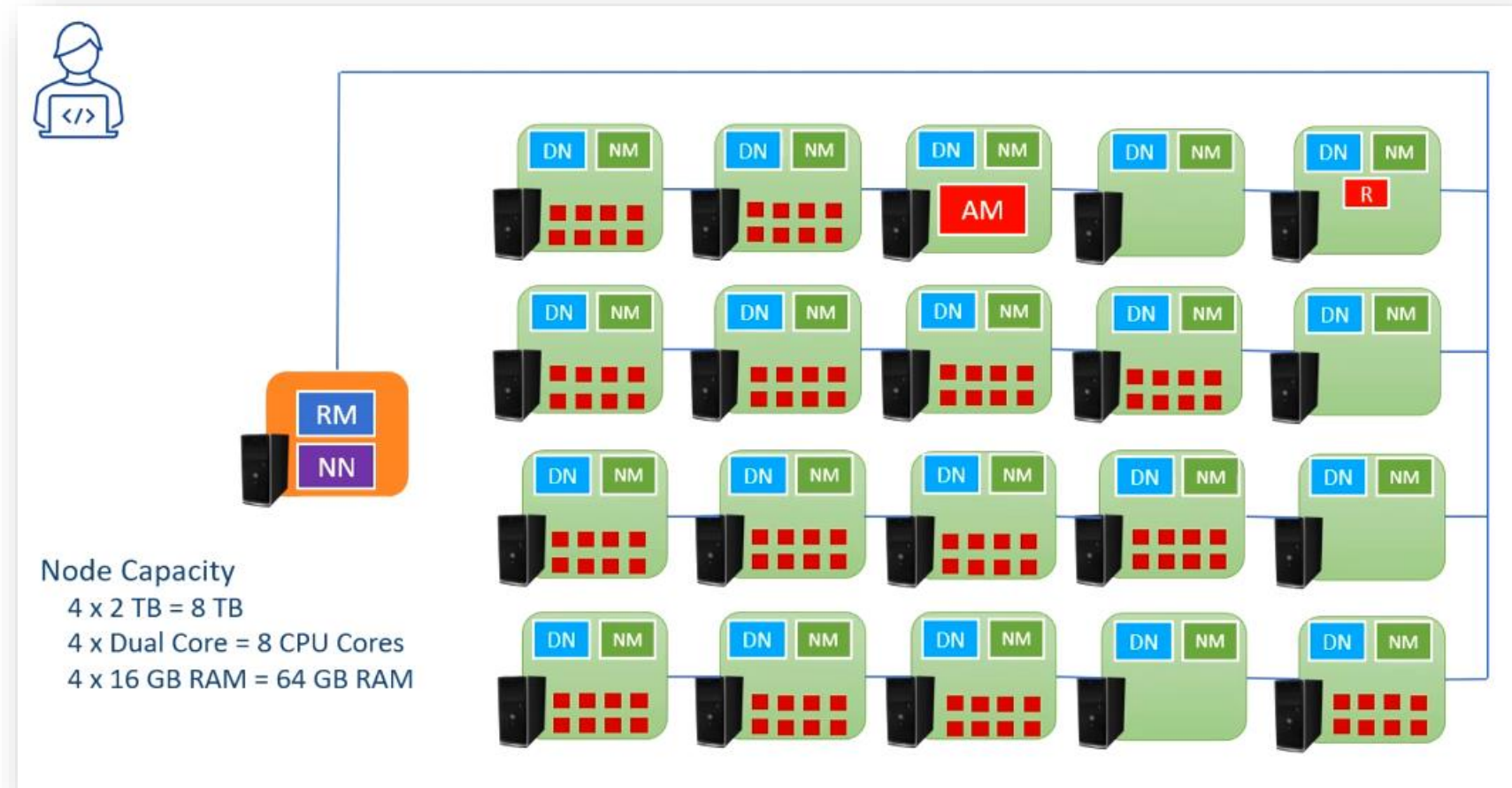
Map/Reduce:

Hadoop offered to solution to both problems we discussed in the previous slide. You can use the Hadoop cluster to store the file. Let's assume you have a 21 node Hadoop cluster. One node becomes the master, and the other 20 nodes are the workers. HDFS runs a name node in the master and a data node on the other workers. YARN runs a Resource Manager on the master, and Node Manager runs on the workers. So we have those services running on the cluster.



Map/Reduce:

You can use HDFS to copy your 20 TB file on this cluster. HDFS will break the file into small 128 MB blocks and spread them across the cluster. So some data nodes will store data blocks, and altogether they can easily store your 20 TB file. Your storage problem is taken care of by the Hadoop cluster. If you need more storage, you can increase the cluster size and add more computers.



Map/Reduce:

Now let us come to the processing time problem. I have broken my logic into two part which you can see in the image below. The first part is known as the Map function. The second part is known as the Reduce function. The old logic was to open the file and count the lines. And the new logic is almost the same as old logic. But the map function opens the file block and counts the lines. And the old logic opens the file and counts the lines.



Problem Statement

Count the lines in the given file

Distributed Solution Pseudocode

```
def map(file_block):  
    open file_block as fb_hd  
    for each t_line in fb_hd.get_line()  
        n_count = n_count + 1  
  
    close fb_hd  
    return n_count  
  
def reduce(list_counts):  
    for each cnt in list_counts  
        total_count = total_count + cnt  
  
print total_count
```

Data File
Format: CSV
Size: 20 TB

Map/Reduce:

I can run the map function on all the data nodes in parallel. This map() function will open each block on the data node and count the lines. End of the execution, I will have the number of lines in the blocks at the given data node. I am counting lines on 14 data nodes in parallel. Everything runs at the same time. And I will get the line counts in 1/14th of the time compared to doing it on a single machine. However, I will have 14 line counts. Each count represents the number of lines on their respective data node.

Problem Statement

Count the lines in the given file

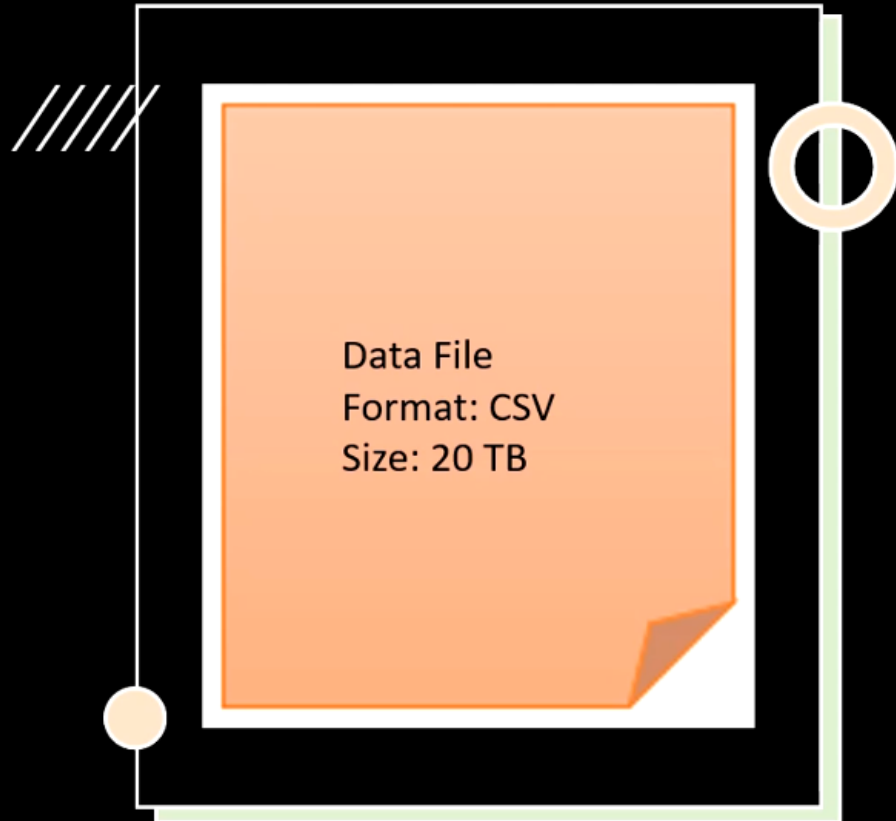
Distributed Solution Pseudocode

```
def map(file_block):  
    open file_block as fb_hd  
    for each t_line in fb_hd.get_line():  
        n_count = n_count + 1
```

```
    close fb_hd  
    return n_count
```

```
def reduce(list_counts):  
    for each cnt in list_counts:  
        total_count = total_count + cnt
```

```
print total_count
```



The diagram shows a large black rectangle representing a data node. Inside it is a smaller white rectangle representing a file block. The file block has an orange border and a folded bottom-right corner. Inside the white rectangle, the text 'Data File Format: CSV Size: 20 TB' is displayed. To the left of the white rectangle, there are four diagonal lines. To the right, there is a green vertical line with a white circle at the top. At the bottom left of the white rectangle, there is a small orange circle.

Data File
Format: CSV
Size: 20 TB

Map/Reduce:

Then, I will start a Reduce function at one node. All the data node will send their counts to the reduce function. The reduce function will receive 14 line counts in an array. So I will look through the array and sum up all the line counts. The reduce function will loop through the list of counts and sum it up. And the sum is the number of lines in the file.

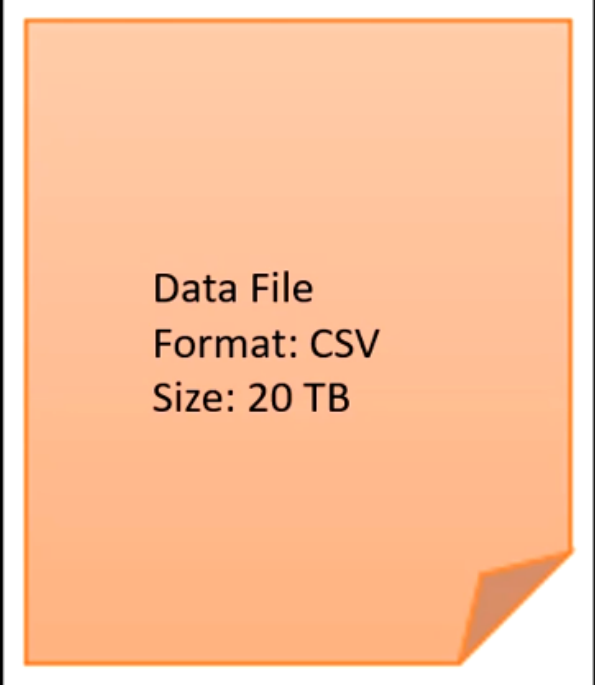
Problem Statement

Count the lines in the given file

Distributed Solution Pseudocode

```
def map(file_block):  
    open file_block as fb_hd  
    for each t_line in fb_hd.get_line():  
        n_count = n_count + 1  
  
    close fb_hd  
    return n_count
```

```
def reduce(list_counts):  
    for each cnt in list_counts:  
        total_count = total_count + cnt  
  
    print total_count
```

A diagram of a data file represented as an orange rectangular card with a folded bottom-right corner. The card is placed on a black background with a white border. To the left of the card are four white diagonal slashes. To the right of the card is a white circle. The text on the card is: Data File, Format: CSV, Size: 20 TB.

Data File
Format: CSV
Size: 20 TB

Map/Reduce:

Here is the summarized context of Map Reduce.

Map Reduce Model

Implement logic in two functions

1. Map Function

- Reads data block
- Applies logic at block level
- Map output is sent to Reduce

2. Reduce Function

- Receives Map output
- Consolidates the results

Hadoop M/R framework implement the map-reduce model.

- YARN manages resource allocation
- HDFS manages data blocks

The three Big Data problems:

1. High Data Volume
2. Variety of Data
3. High speed

Google was the first company to realize the big data problem. And they were also the first company to develop a viable solution and establish a commercially successful business around it.

Google had four main problems to solve which are listed below.

They were creating a search engine, and the first thing they wanted to do was to discover and crawl the web pages over the Internet and capture the content and metadata. We now categorize these problems as data collection and data ingestion.

Once the webpage-related data is collected, they want to store and manage it. So the next problem was to store and manage hundreds of petabytes of data.

The next problem was to get the computation power for processing those massive volumes. They wanted to apply the PageRank algorithm to the received data and create an index. So, the third problem was to get the required processing power.

Finally, the last problem was organizing and storing the outcome of the processing. In Google's case, the result was the index.

They wanted to keep it in a random-access database to support high-speed queries by the Google Search Engine application.

Google successfully solved all the four problems, and they were generous enough to reveal their solution to the rest of the world in a series of white papers. Google published the first whitepaper in 2003, and it talked about solving the Data Storage and Management problem. They termed the solution as Google File System (GFS).

The second whitepaper was published by Google in 2004 and talked about the Data Processing and Transformation problem. They termed it as MapReduce (MR) programming model.



1. Google File System – 2003

<https://ai.google/research/pubs/pub51>

2. MapReduce – 2004

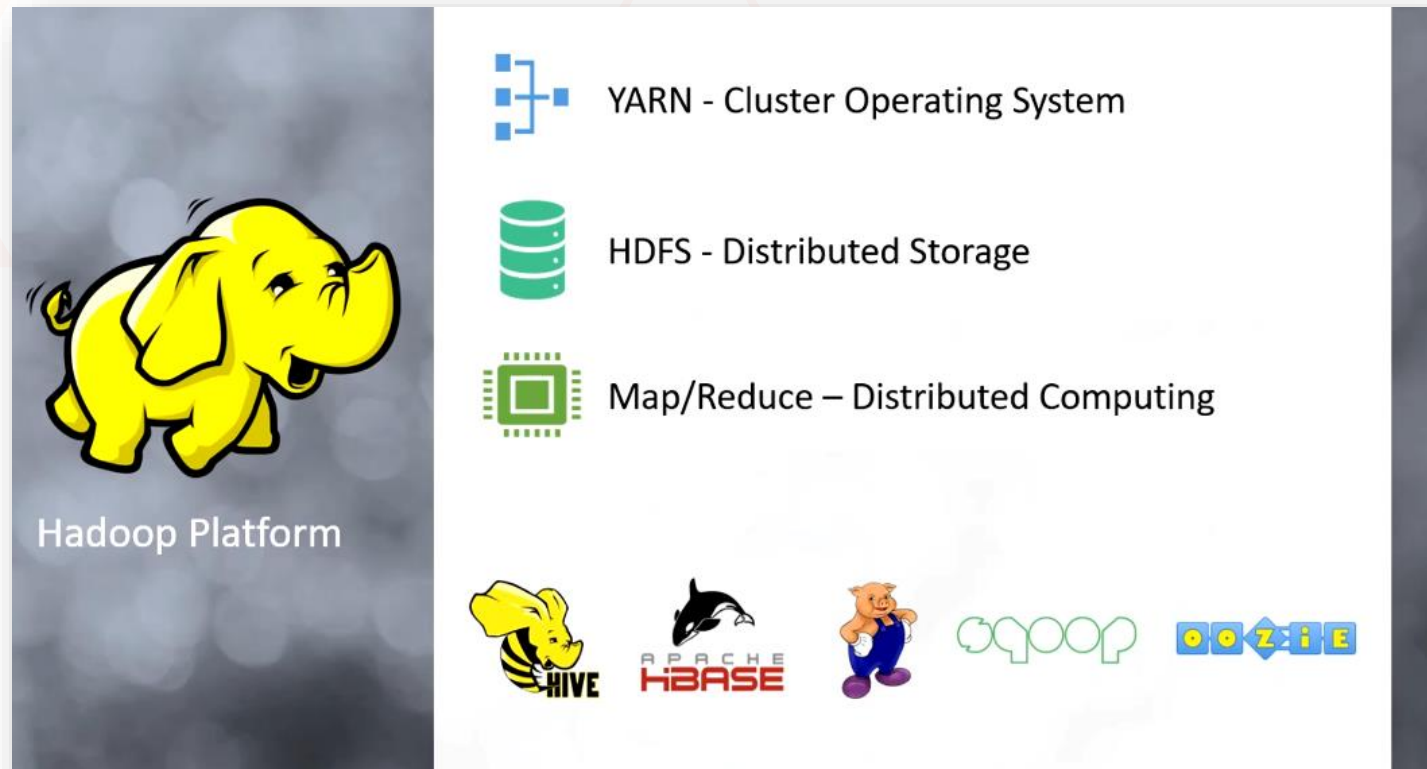
<https://ai.google/research/pubs/pub62>

The open-source community well appreciated these whitepapers, and they formed the basis for the design and the development of a similar open-source implementation – The Hadoop. The open-source community implemented the GFS as a Hadoop Distributed File System – HDFS. They applied Google MR as the Hadoop MapReduce programming framework.

Hadoop grabbed immense attention and popularity among organizations and professionals.

Since the development of Hadoop, there have been many other solutions developed over the Hadoop platform and open-sourced by various organizations.

Some of the most widely adopted systems were Pig, Hive, and HBase.



Apache Hive was one of the most popular components of Hadoop. Hive offered the following core capabilities on the Hadoop Platform:

1. Create Databases, Tables, and Views
2. Run SQL Queries on the Hive Tables

So Hive simplified using Hadoop. Application developers struggled to solve data processing problems using Map Reduce. Hive came to the rescue. It allowed us to create databases and tables using DDL Statement.

Then they also allowed us to use SQL queries on the table.

The majority of the development workforce was familiar with the RDBMS, and they already knew SQL.

So using SQL was easy to adopt.

Hive SQL engine internally translated SQL queries into M/R programs.

But application developers were saved from writing Map Reduce code in Java.

Hadoop as a platform and Hive as a Hadoop database became very popular.

But we still had the following problems which needed improvements:

1. Performance
2. Ease of development
3. Language support
4. Storage
5. Resource Management

Apache Spark comes to rescue.

Entering Apache Spark

Advantages over Hadoop

1. Performance
 - 10 to 100 times faster than Hadoop M/R
2. Ease of development
 - Spark SQL
 - High performance SQL Engine
 - Composable Function API
3. Language support
 - Java, Scala, Python and R
4. Storage
 - HDFS Storage
 - Cloud Storage
5. Resource Management
 - YARN, Mesos, Kubernetes



Runs in two setups

1. With Hadoop (Data Lake)
2. Without Hadoop (Lakehouse)

Spark exists on two kinds of platforms:

1. On Hadoop Platform - Data Lake
2. On Cloud Platform - Lakehouse

We use the Hadoop platform as the Data Lake platform, and the primary developer technology on Hadoop Data Lake is now Apache Spark.

Map/Reduce Framework is gone away forever, and Hive is also losing its place for Spark SQL.

The Cloud platforms are more popular these days.

So the idea of Hadoop Data Lake is now advanced and modernized with a new name of Lakehouse on the cloud platforms.

The driving force behind Cloud Lakehouse is the Databricks Spark platform. So Spark is again the primary developer technology for Lakehouse.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com